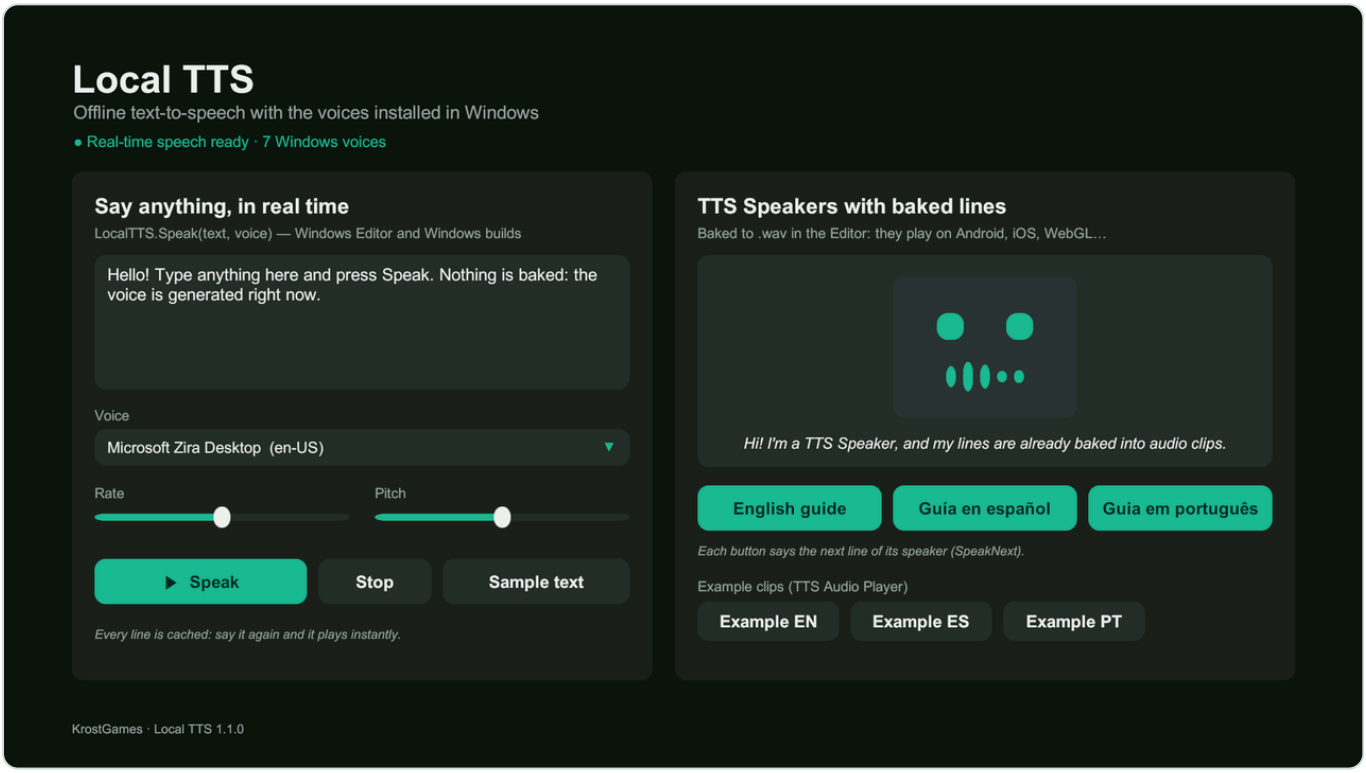
Local TTS

Texto a voz offline para Unity con las voces que ya trae Windows

Habla en tiempo real u hornea líneas en clips .wav que suenan en cualquier plataforma.
Sin nube, sin claves de API, sin tokens.

Versión 1.1.0 · Unity 2022.3 LTS o superior · KrostGames



Contenido

1. Qué hace
2. Requisitos
3. Instalación
4. Inicio rápido: tres formas de usarlo
5. ¿Tiempo real u horneado? Qué hace cada plataforma
6. La ventana Generator
7. Componente TTS Speaker
8. Line Library
9. Perfiles de voz
10. Programación
11. Archivos por lotes
12. Instalar más voces
13. Estructura de carpetas
14. Actualizar desde la 1.0
15. Preguntas frecuentes y solución de problemas

Qué hace

- **Voz en tiempo real, sin hornear:** `LocalTTS.Speak("¡Hola!")` sintetiza y reproduce la línea al momento, en el Editor de Windows y en builds de Windows. Las líneas repetidas salen de una caché en memoria y suenan al instante.
- **Hornea una vez, suena en todas partes:** convierte cualquier línea en un AudioClip `.wav` estándar que suena en Android, iOS, WebGL, consolas, macOS o Linux.
- **Componente TTS Speaker:** escribe las líneas en el Inspector, escúchalas con ▶, hornéalas con un clic y llama a `Speak()` desde botones de UI, interacciones XR, Timeline o código. La voz sale por el AudioSource del GameObject, así que puede ser 3D y pasar por un mixer.
- **Line Library:** las líneas que tu código dice en Play Mode se registran solas, listas para hornear, y así el mismo código también habla en plataformas sin síntesis en tiempo real.
- **Ventana Generator:** escribe, escucha desde memoria, guarda `.wav`, genera por lotes desde `.txt` / `.csv` y arrastra clips del historial a cualquier campo AudioClip.
- **Todas las voces de Windows:** las clásicas SAPI 5 "Desktop" y las voces OneCore de Windows 10/11 (las que agregas en Configuración > Voz), en cualquier idioma que instales.
- **Velocidad, tono y volumen** por línea, perfiles de voz compartibles entre speakers y SSML para control fino.
- **Interfaz del Editor en inglés, español y portugués.**

Requisitos

Unity	2022.3 LTS o superior (probado en Unity 6.3, 6.4 y 6.5)
Generar voz	Windows 10 u 11 (Editor o build de Windows). Usa Windows PowerShell 5.1, incluido en Windows
Voces	Al menos una voz de Windows instalada (Windows trae una o más para su idioma)
Reproducir clips horneados	Cualquier plataforma que soporte Unity

No se necesita conexión a internet en ningún momento.

Instalación

1. Importa el paquete (Assets > Import Package > Custom Package..., o desde *My Assets* en el Package Manager). Se instala en `Assets/KrostGames/LocalTTS`.
2. Puedes mover la carpeta a cualquier lugar dentro de `Assets/`: las herramientas encuentran sus archivos solas.
3. Abre **Tools > Local TTS > Generator**. El indicador de estado se pone verde ("Listo · N voces") cuando el motor de voz arrancó.

Inicio rápido: tres formas de usarlo

1. Ventana Generator (sin código)

Tools > Local TTS > Generator. Escribe una línea, elige una voz, pulsa ▶ **Escuchar** (suena desde memoria, no guarda nada) y **Generar .wav** para guardarla como asset. Con *Asignar al objeto seleccionado* activo, el clip va

directo al TTS Speaker, TTS Audio Player o Audio Source seleccionado.

2. Componente TTS Speaker (Inspector)

GameObject > Audio > TTS Speaker (o *Add Component > KrostGames > Local TTS > TTS Speaker*). Escribe las líneas, pulsa ▶ en cualquiera para oírla y **Hornear faltantes** cuando las quieras en builds que no son de Windows. Conecta `Speak()`, `SpeakNext()`, `SpeakLine(int)` o `Speak(string)` a cualquier UnityEvent, como un Button de UI o un interactable de XR.

3. Una línea de código

```
using KrostGames.TTS;

LocalTTS.Speak("¡Hola! Esta línea se generó ahora mismo.");
```

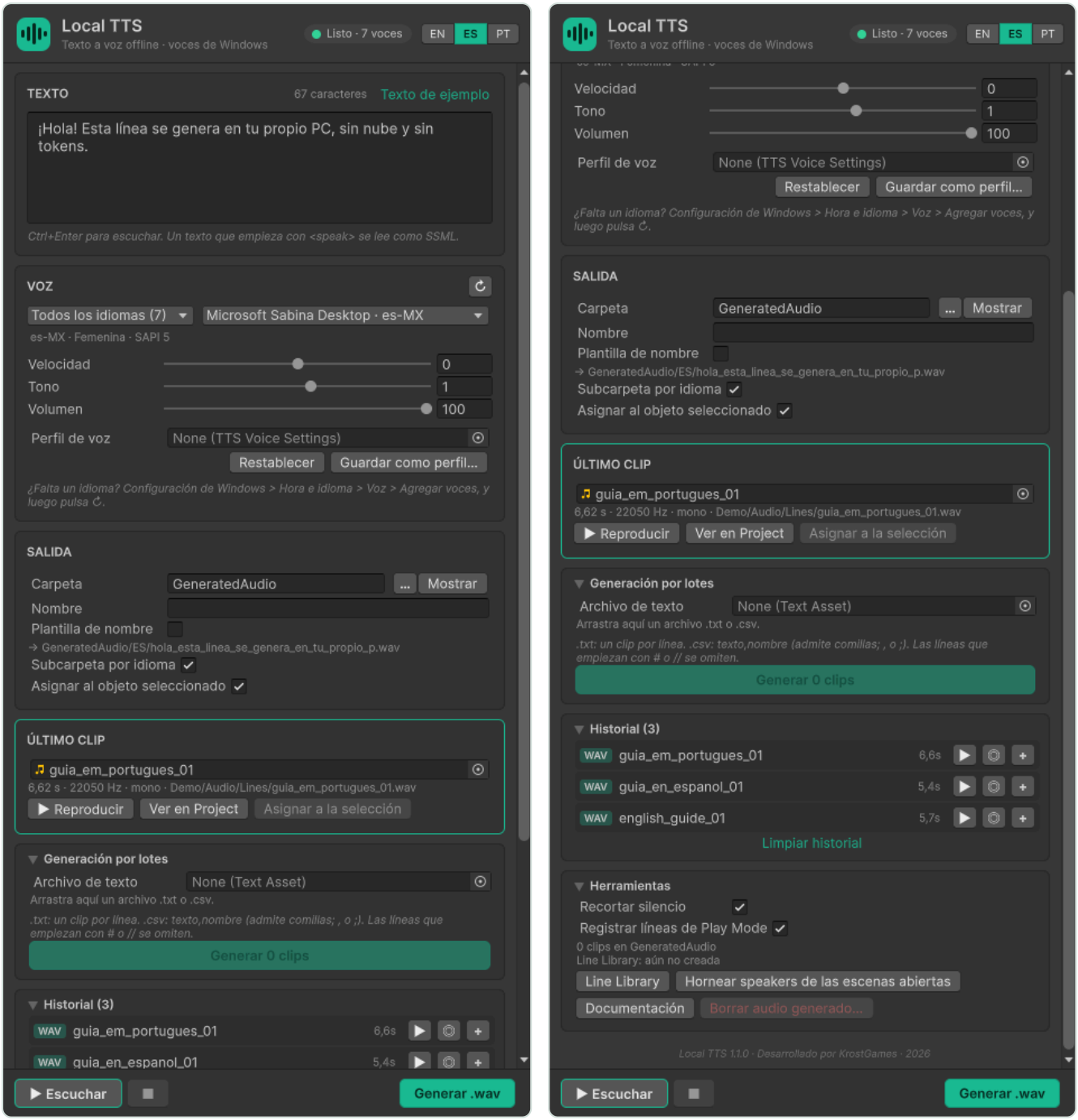
Nada más: sin componentes ni configuración. Las líneas se encolan en un speaker 2D compartido.

¿Tiempo real u horneado? Qué hace cada plataforma

	Editor de Windows	Build de Windows	Android, iOS, WebGL, consolas, macOS, Linux
Reproducir clips horneados	✅	✅	✅
Voz en tiempo real (`LocalTTS.Speak` , líneas de speaker sin clip)	✅	✅	— usa el clip horneado
Generar / hornear `.wav`	✅	—	—

Un TTS Speaker en modo **Auto** hace lo correcto en cada plataforma: reproduce el clip horneado cuando coincide con el texto y la voz actuales de la línea, sintetiza en tiempo real cuando no coincide (Windows) y, en las demás plataformas, usa cualquier clip horneado disponible. Su Inspector te dice cuántas líneas quedarían en silencio en el build target activo.

La ventana Generator

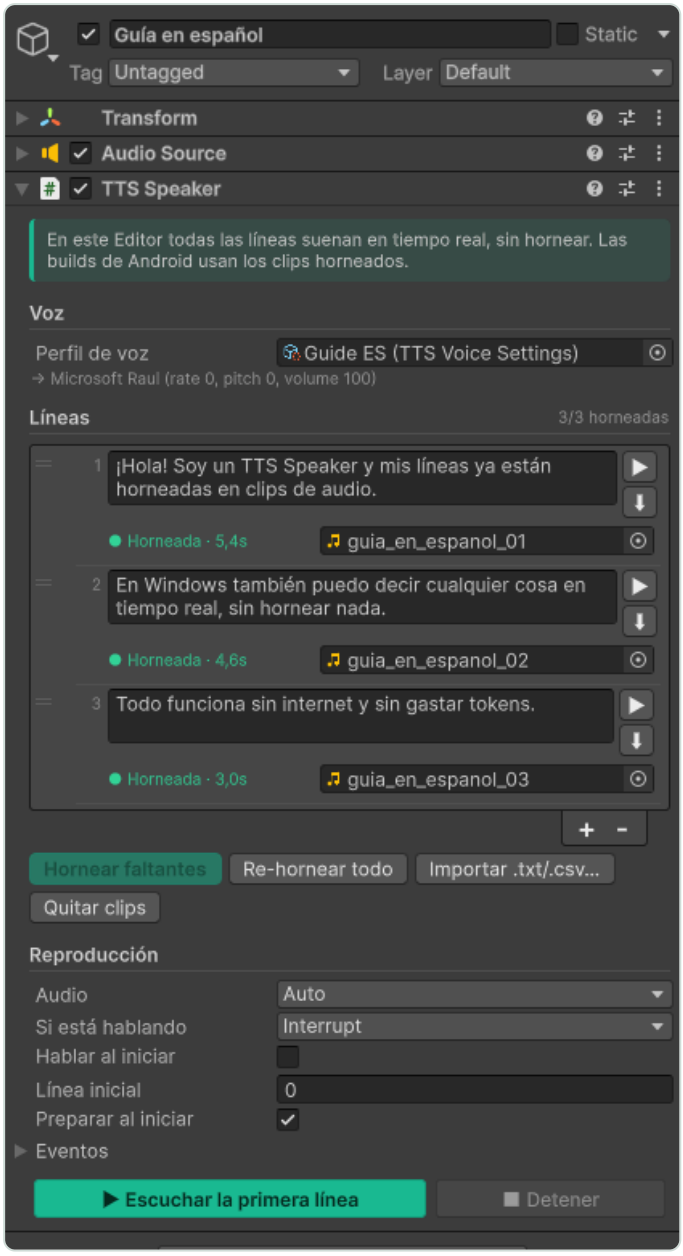


Tools > Local TTS > Generator: texto, voz y salida (izquierda); último clip, lotes, historial y herramientas (derecha).

Zona	Qué hace
Cabecera	Estado del motor (haz clic para reintentar tras un error) e idioma de la interfaz (EN / ES / PT).
Texto	La línea a decir. `Ctrl+Enter` la escucha. Un texto que empieza con `<speak>` se lee como SSML.
Voz	Filtro de idioma, voces instaladas (SAPI y OneCore), velocidad, tono y volumen. Carga un *Voice Profile* o guarda los ajustes actuales como uno nuevo. ↻ recarga la lista tras instalar voces.
Salida	Carpeta de salida (por defecto `GeneratedAudio/`), nombre de archivo opcional, plantilla de nombre (`{lang}` `{id}` `{name}` `{voice}`), subcarpeta por idioma y asignación a la selección.
Último clip	Reproducir, ver en Project, asignar a la selección o arrastrar la tarjeta a cualquier campo AudioClip.
Generación por lotes	Suelta un `.txt` o `.csv` y genera todas las líneas. El Editor sigue respondiendo; Cancelar conserva los clips ya terminados.
Historial	Los últimos 20 clips: reproducir, localizar, asignar o arrastrar.
Herramientas	Recortar silencio, registrar líneas de Play Mode, Line Library, hornear todos los speakers de las escenas abiertas, documentación y mover todo el audio generado a la papelera.

Nombres de archivo: con un nombre, un archivo existente con ese nombre se **reemplaza y conserva sus referencias** (útil para regrabar una línea); sin nombre, se toma del texto y nunca sobrescribe nada.

Componente TTS Speaker

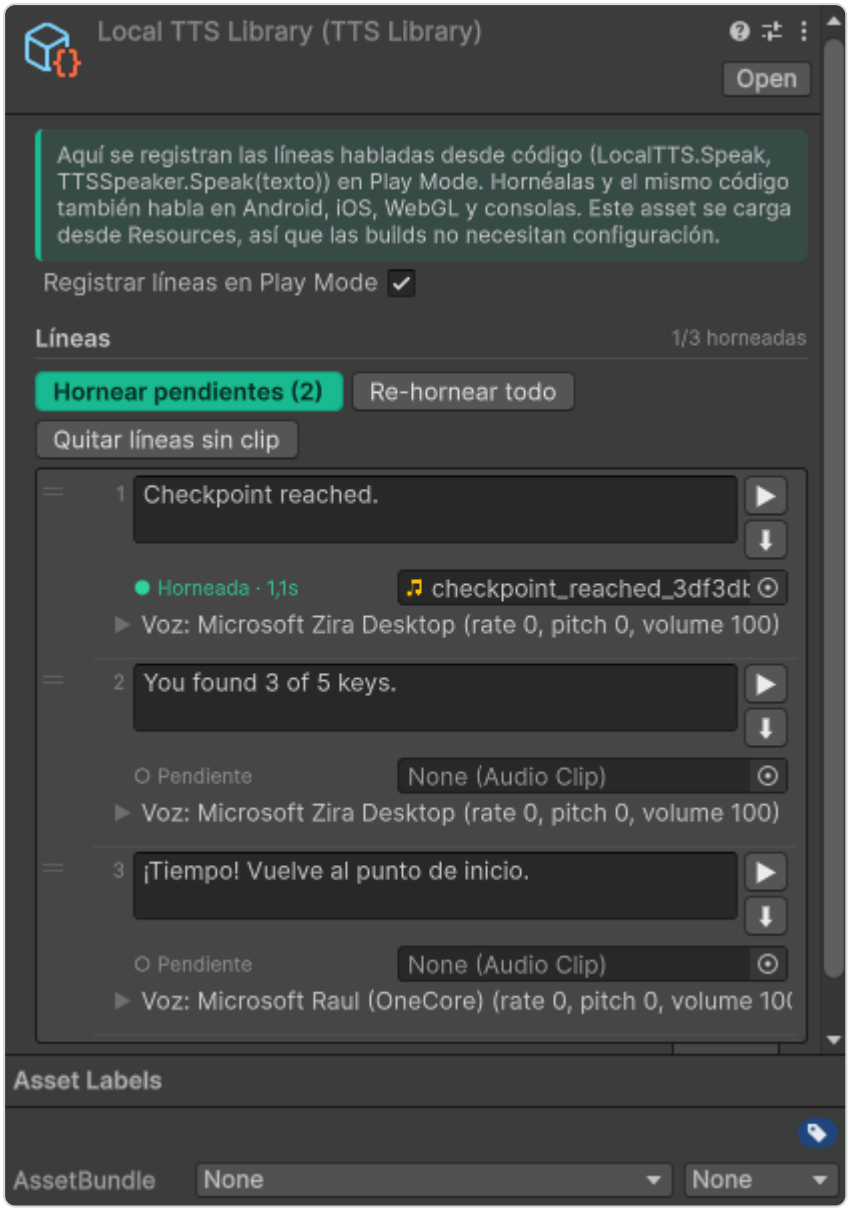


Un TTS Speaker con un perfil de voz y tres líneas horneadas.

Ajuste	
Voice Profile	Voz compartida opcional. Si está vacío se usan los ajustes de voz del propio componente (selector de voces instaladas, idioma, velocidad, tono, volumen y ▶ para probar).
Líneas	El texto de cada línea, su clip horneado y su estado: ● Horneada, ▲ Desactualizada (cambió el texto o la voz), ○ Solo tiempo real. Reordena arrastrando. **Hornear faltantes**, **Re-hornear todo**, **Importar .txt/.csv**, **Quitar clips**.
Audio	*Auto* (recomendado), *Realtime Only* (siempre sintetiza, Windows), *Baked Only* (exactamente lo que hacen las builds que no son de Windows).
Si está hablando	*Interrupt*, *Queue* o *Ignore*: qué pasa con una línea nueva mientras otra suena.
Hablar al iniciar / Línea inicial	Decir una línea cuando arranca la escena.
Preparar al iniciar	Arranca el motor y sintetiza las líneas en segundo plano para que la primera no tenga demora (Windows).
Eventos	`On Speech Started (string)` y `On Speech Finished (string)`. Finished se lanza cuando una línea suena hasta el final (no con `Stop()`).

Los clips horneados se nombran a partir del texto y de la voz: hornear dos veces la misma línea reutiliza el archivo y las líneas idénticas comparten un solo clip.

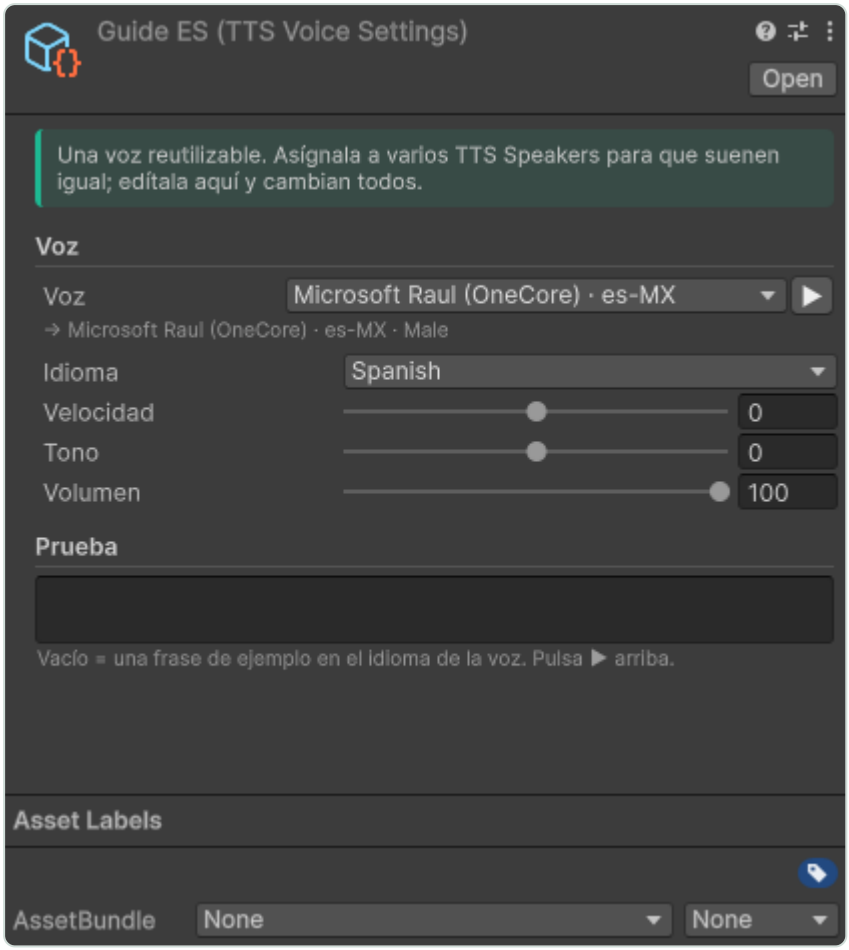
Line Library



La Line Library: líneas registradas desde código, una horneada y dos pendientes.

Las líneas dichas desde código (`LocalTTS.Speak` , `TTSSpeaker.Speak(string)`) durante Play Mode se agregan a la **Line Library** (`GeneratedAudio/Resources/LocalTTSLibrary.asset` , se crea al usarla por primera vez). Ábrela con **Tools > Local TTS > Line Library** y pulsa **Hornear pendientes**. Desde ese momento el mismo código encuentra esos clips en cualquier plataforma. Como el asset está en una carpeta `Resources` , las builds no necesitan configuración. Al hacer una build para una plataforma sin voz en tiempo real, un aviso en la consola indica cuántas líneas de la librería siguen sin hornear. El registro se puede desactivar en Herramientas del Generator o en la propia librería.

Perfiles de voz



Un asset Voice Profile con el selector de voces instaladas y ▶ para probar.

Assets > Create > Local TTS > Voice Profile. Un perfil guarda voz, idioma, velocidad, tono y volumen. Asígnalo a varios speakers ("Narrador", "Robot", "Guía") y edítalo una vez para cambiarlos todos. Su Inspector tiene el mismo selector de voces y una frase de prueba.

Programación

Todo está en el namespace `KrostGames.TTS`. Llámalo desde el hilo principal de Unity.

```
using KrostGames.TTS;
using UnityEngine;

public class Ejemplos : MonoBehaviour
{
    public TTSSpeaker guia;
    public AudioSource sfx;
    public Transform boca;

    async void Start()
    {
        // Elige la voz por nombre completo o parcial; el idioma se usa si esa voz no está instalada.
        LocalTTS.DefaultVoice = new TTSVoiceOptions("Raul", TTSLanguage.Spanish, rate: 1, pitch: -1);
        LocalTTS.Prewarm();                              // arranca el motor antes (pantallas de carga)

        LocalTTS.Speak("¡Bienvenido!");                  // speaker 2D compartido, las líneas se encolan

        AudioClip clip = await LocalTTS.SynthesizeAsync("Puntos: 120");   // este clip es tuyo
        sfx.PlayOneShot(clip);

        foreach (TTSVoice voz in await LocalTTS.GetVoicesAsync())
            Debug.Log(voz);                              // "Microsoft Sabina Desktop (es-MX)"...

        guia.OnSpeechFinished.AddListener(texto => Debug.Log("Terminó: " + texto));
        guia.SpeakLine(0);
    }

    void Update()
    {
        // Volumen (0..1) de lo que suena ahora: mueve una boca, una luz o un medidor de UI.
        boca.localScale = new Vector3(1f, 0.2f + guia.Loudness, 1f);
    }
}
```

API	
`LocalTTS.Speak(texto, voz = null)`	Habla con el speaker compartido (clip horneado de la Line Library si existe; si no, tiempo real).
`LocalTTS.SynthesizeAsync(texto, voz)`	Nuevo `AudioClip` con la línea (Windows). `SynthesizeDataAsync` devuelve el PCM.
`LocalTTS.Prepare(texto, voz)` / `Prewarm()`	Sintetiza en segundo plano / arranca el motor.
`LocalTTS.GetVoicesAsync()`, `Voices`, `ResolveVoice(opciones)`	Voces instaladas y cuál eligen unas opciones.
`LocalTTS.IsRealtimeSupported`, `State`, `StateChanged`, `LastError`	Estado del motor.
`LocalTTS.DefaultVoice`, `TrimSilence`, `CacheSize`, `StopAll()`, `Shutdown()`	Ajustes globales y control.
`TTSSpeaker.Speak()`, `SpeakNext()`, `SpeakLine(i)`, `Speak(texto)`, `Speak(texto, voz)`, `Stop()`, `Pause()`, `Resume()`, `Prepare()`	Control del speaker (todo usable desde UnityEvents).
`TTSSpeaker.IsSpeaking`, `IsPreparing`, `CurrentText`, `CurrentLine`, `Loudness`, `Lines`, `Voice`, `Mode`, `WhenBusy`	Estado y ajustes del speaker.

Scripts de Editor (`KrostGames.TTS.EditorTools`): `WindowsTTSEngine.GenerateAsync(texto, voz, nombre, carpeta)`, `BakeAsync(texto, voz)` y `BatchGenerateAsync(...)` escriben assets `.wav`. Los métodos de la 1.0 (`Generate`, `BatchGenerate`, `LivePreview`, `StopLivePreview`, `GetVoicesForLanguage`...) siguen disponibles.

SSML: un texto que empieza con `<speak` se envía como SSML, por ejemplo `<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="es-MX">Espera<break time="500ms"/> ¡ya!</speak>`.

Archivos por lotes

- **`.txt`:** un clip por línea. Se omiten las líneas vacías y las que empiezan con `#` o `//`. Las comas dentro de una línea son parte del texto.
- **`.csv`:** `texto,nombre de archivo` por fila; el nombre es opcional. Pon entre comillas los campos que tengan comas (`"Hola, mundo",saludo`) y escribe las comillas como `""`. También funciona el punto y coma (Excel en español). Una primera fila `text,filename` o `texto,nombre` se trata como encabezado.

Instalar más voces

1. En Windows: **Configuración > Hora e idioma > Voz > Administrar voces > Agregar voces** (en Windows 10: *Configuración > Hora e idioma > Voz > Agregar voces*).
2. Elige el idioma (por ejemplo *English (United States)*, *Português (Brasil)*) e instálalo.
3. Pulsa ↻ en el Generator. No hace falta reiniciar Unity.

Windows tiene dos familias de voces y se listan las dos: **voces SAPI 5 "Desktop"** (por ejemplo *Microsoft Sabina Desktop*) y **voces OneCore** (por ejemplo *Microsoft Raul*, mostrada con "(OneCore)"). Muchos idiomas instalan ambas.

Estructura de carpetas

```
Assets/KrostGames/LocalTTS/
├── README.md · LEEME.md · CHANGELOG.md · LICENSE.md · package.json
├── Documentation/    LocalTTS_Manual_EN.pdf · LocalTTS_Manual_ES.pdf
├── Runtime/          KrostGames.LocalTTS: LocalTTS, TTSSpeaker, TTSVoiceOptions, TTSVoiceSettings,
│                     TTSLibrary, TTSAudioPlayer, Engine/ (proceso de voz, WAV)
├── Editor/           KrostGames.LocalTTS.Editor (solo Editor): Generator, inspectores, horneado
├── Demo/             LocalTTS_Demo.unity · Scripts/ · Voices/ · Audio/ (líneas demo, ejemplos)
└── GeneratedAudio/   Salida por defecto: EN/, ES/, PT/… y Resources/LocalTTSLibrary.asset
```

Los ajustes del proyecto se guardan en `ProjectSettings/KrostGames.LocalTTS.asset` (compartidos con tu equipo) y el estado personal del Generator en `UserSettings/`.

Actualizar desde la 1.0

- Los namespaces ahora son `KrostGames.TTS` (runtime) y `KrostGames.TTS.EditorTools` (Editor). Cambia `using KrostGames.LocalTTS.Runtime.Core;` por `using KrostGames.TTS;`.
- Los ensamblados se llaman `KrostGames.LocalTTS` y `KrostGames.LocalTTS.Editor`.
- Los menús ahora están en **Tools > Local TTS** y las instalaciones nuevas van a `Assets/KrostGames/LocalTTS`.
- Los clips de ejemplo se movieron a `Demo/Audio/Examples/`.
- `TTSAudioPlayer`, `TTSVoiceSettings` y los métodos 1.0 de `WindowsTTSEngine` siguen funcionando. La lista completa está en CHANGELOG.md.

Preguntas frecuentes y solución de problemas

El indicador dice "Solo Windows". La síntesis de voz necesita Windows. Los clips horneados en Windows suenan en todas partes, así que un compañero con macOS o Linux puede usarlos.

No hay voces para mi idioma. Instala una voz (ver arriba) y pulsa ↻.

¿Puedo usar voz en tiempo real en Android / Meta Quest / iOS? En tiempo real no: esas plataformas no tienen las voces de Windows. Hornea las líneas (TTS Speaker > *Hornear faltantes*, o la Line Library) y suenan ahí. El Inspector del speaker avisa de las líneas que quedarían en silencio en el build target activo.

"Could not start Windows PowerShell" o el motor queda en Error. La voz en tiempo real usa un proceso oculto `powershell.exe`. Algunas políticas de empresa lo bloquean; los clips horneados siguen funcionando. No hace falta cambiar la política de ejecución de PowerShell: el asset no ejecuta archivos de script.

Mi antivirus preguntó por PowerShell. Es el proceso del motor de voz mencionado arriba; solo usa las APIs de voz de Windows y se cierra con Unity o con tu juego.

La voz suena robótica. La calidad depende de la voz de Windows instalada. Prueba las voces OneCore de *Agregar voces* y ajusta velocidad y tono.

La primera línea tarda un poco. Arrancar el motor toma uno o dos segundos la primera vez. Llama a `LocalTTS.Prewarm()` o deja activo *Preparar al iniciar*.

¿Necesita internet o cuesta algo por uso? No. Todo corre en el PC local con las voces de Windows.

Local TTS 1.1.0 · Desarrollado por **KrostGames** · 2026